

Analisi Approfondita dei Cambiamenti: Confronto Evolutivo tra Sprint 0 e Sprint 1 nel Modello QAK

Davide Chirichella, Gabriele Doti, Daniele
Maccagnan

Ingegneria dei Sistemi Software, A.A. 2025/2026
Alma Mater Studiorum . Università di Bologna

July 06, 2026

1. Obiettivi dello Sprint 1

Obiettivo dello Sprint 1 è quello, partendo dall'architettura e dalle considerazioni analitiche maturate nello Sprint 0, di analizzare, progettare e realizzare un prototipo eseguibile del core-business del sistema da poter sottoporre a verifica formale e mostrare al committente.

Nello specifico, in ottica di **Separation of Concerns** e sulla base delle priorità operative dei requisiti, l'analisi si concentra sulla correttezza algoritmica e comportamentale dell'orchestratore principale (`cargoservice`). Bisogna considerare che le funzionalità trattate in questa fase presuppongono l'interazione con sensori fisici (sonar), attuatori (LED, marker laser) e robot che non sono ancora stati sviluppati o integrati. Per tale ragione, nello Sprint 1 verranno impiegati dei **componenti mock** progettati ad hoc per replicare in modo deterministico e ripetibile il protocollo di interazione dei sottosistemi mancanti.

1.1. Sottoinsieme di Requisiti Considerati

Come stabilito nello Sprint 0, il lavoro si focalizza sul sottoinsieme di requisiti ad alta priorità che regolano il ciclo di ammissione, etichettatura e deposito del container:

- **Requisiti `cargoservice` :**
 - Deve poter accogliere le richieste di carico (`load_request`) inviate dai clienti (`ioport`).
 - Deve poter rifiutare la richiesta se tutti gli slot sono occupati (`hold_full`), se il sistema è in stato di allarme (`!ServiceWorking`) o se l'area di ingresso è già ingombra da un altro container (`IOPortOccupied`).
 - In caso di richiesta accettata (`load_accepted`), deve riservare una cella in stiva (`get_slot`), associare lo slot e comunicarne il nome al cliente.
 - Durante la gestione del carico, deve disabilitare l'accettazione immediata di altre richieste (instradandole verso un rinvio `load_retrylater`).
 - Deve attendere la rilevazione fisica del container all'interfaccia di ingresso, coordinare le azioni di prelievo, etichettatura e deposito con il robot e interrompere ogni attività in caso di anomalie ambientali o guasti.

2. Analisi del Problema e Nuova Architettura Logica

Il passaggio dal modello dello **Sprint 0** (`/sprint0/src/cargosystem.qak`) al prototipo dello **Sprint 1** (`/sprint1/prototype/codice_con_tutti_i_componenti.qak`) evidenzia l'applicazione del principio di “**zooming architetturale**”: si transita da una vista astratta di dominio a una specifica implementativa pienamente reattiva.

2.1. Analisi della Topologia di Rete (Contesti QAK)

Una prima fondamentale indagine riguarda la disposizione degli attori nei contesti di esecuzione. A questo punto si aprono due possibilità: mantenere i 4 contesti distribuiti ipotizzati nello Sprint 0 (`ctxcargoservice` , `ctxioport` , `ctxrobot` , `ctxdevices`) oppure raggruppare i componenti in un contesto locale unico.

PARA-METRO	SPRINT 0 (ANALISI)	SPRINT 1 (PRO-TOTIPO)	MOTIVAZIONE IN-EGGERISTICA
Topologia	4 Contesti elementari distanti tra loro	1 Contesto Unico (<code>ctxcargoservice :8050</code>)	Focalizzazione sulla correttezza logica della FSM senza introdurre complessità di rete premature.
Meccanismo di Comunicazione	TCP/IP tra nodi di rete separati	Scambio messaggi locale (in memoria, stessa JVM)	Elimina latenza e overhead di serializzazione, rendendo il debug formale e immediato.
Preparazione per Sprint 2	N/A	Architettura multi-contesto già archiviata in <code>uTils/prototype/</code>	La separazione in contesti fisici distribuirà gli attori sui nodi reali nel prossimo Sprint.

La scelta di adottare un **Contesto Unico nello Sprint 1** risponde a un criterio rigoroso: isolare la verifica logico-algoritmica da problematiche infrastrutturali. Se si fossero distribuiti subito i mock su socket TCP separati, eventuali fallimenti durante i collaudi avrebbero potuto essere causati da problemi di rete (firewall, porte occupate, serializzazione) anziché da errori nella Macchina a Stati Finiti. La topologia distribuita, già formalizzata e archiviata nella libreria `uTils/prototype/`, costituisce la base evolutiva pronta per il deployment fisico nello Sprint 2.

2.2. Analisi delle Alternative e Uso del Linguaggio QAK

La progettazione del sistema ha tratto un vantaggio decisivo dall'adozione del meta-modello QAK. L'abstraction gap colmato dal DSL alleggerisce lo sviluppo di molti dettagli già gestiti nativamente dal runtime, come la gestione delle code di messaggi, il multithreading e lo scambio asincrono di eventi.

- **Gestione dell'Accodamento Richieste:** Dall'incontro con il committente è emerso che le richieste di carico inviate durante una fase operativa debbano poter essere accodate o rinviate in modo ordinato. La semantica a messaggi di QAK assorbe questa complessità nativamente tramite le code di ricezione degli attori.
- **Interazione Client-Servizio:** Per vincolare il cliente all'esito della richiesta di carico, si è mantenuto il paradigma **Request/Reply** rigoroso (`load_request` to `load_accepted` / `load_retrylater` / `load_refused`). Alternative come l'uso di semplici dispatch o coppie dispatch/event avrebbero reso la correlazione della risposta inutilmente frammentata.

3. Progettazione e Scelte Implementative (`cargoservice`)

La progettazione dell'orchestratore si basa sul principio di **Single Point of Control**: `cargoservice` coordina tutte le fasi di ammissione, interroga la stiva e gestisce le movimentazioni senza delegare la logica di controllo ad altri entità. Di seguito vengono analizzate le scelte progettuali attuate nel passaggio da Sprint 0 a Sprint 1.

3.1. 1. Sdoppiamento dello Stato di Attesa (`work` → `disengaged` / `engaged`)

Nello Sprint 0 il sistema sosta in un generico stato `work`. Per aderire fedelmente alla terminologia contrattuale (**“considera il sistema come engaged quando gestisce un carico... altrimenti disengaged”**), lo stato `work` è stato eliminato in favore di due stati espliciti:

- `disengaged` : Sistema in inattività logica, LED spento e pronto a ricevere nuove richieste.
- `engaged` : Sistema impegnato, LED in lampeggio e precondizioni di ammissione attive.

3.2. 2. Guardia Booleana sulle Precondizioni di Carico

In fase di ammissione (`handle_load_request`), le regole decisionali abbozzate a parole nello Sprint 0 sono state tradotte in un'asserzione di guardia valutata deterministicamente:

```
if [# CargoState = "engaged" || !ServiceWorking || IOPort0occupied #] {  
  replyTo load_request with load_retrylater : loadRetryLater(none)  
} else {  
  request hold -m get_slot : getSlot(none)  
}
```

Se l'impianto è occupato, in allarme o la pedana non è sgombra, la richiesta viene deviata senza bloccare l'attore.

3.3. 3. Srotolamento Asincrono (“Unrolling”) del Ciclo Operativo

Nello Sprint 0 l'intero processo di movimentazione, etichettatura e stoccaggio era compresso in commenti inline. Nello Sprint 1, per rispettare la natura non bloccante degli attori, questo flusso è stato srotolato in **7 stati sequenziali e reattivi**:

1. `wait_for_slot` : Attesa asincrona della conferma di disponibilità da parte della stiva (`hold`).
2. `accept_request` / `refuse_request` : Smistamento finale della risposta verso la `ioport`.
3. `handle_sonar` : Ricezione degli eventi `sonardata` per certificare il posizionamento fisico del container.

4. `do_robot_job` : Richiesta di prelievo e trasporto al robot verso la zona di etichettatura (`slot5`).
5. `mark_container` : Interrogazione al `markerdevice` per l'applicazione del codice laser.
6. `move_to_reserved_slot` : Ordine finale al robot per il deposito nella cella riservata.
7. `finish_job` : Risoluzione della transazione, spegnimento del LED e ripristino di `disengaged` .

3.4. 4. Gestione degli Interrupt e Routing Asincrono (`returnToState`)

Un punto aperto di fondamentale importanza riguarda la gestione di eventi asincroni improvvisi, come gli allarmi del sonar (`set_service_status` con payload `outofservice` / `working` o variazioni di `sonardata`). L'attore deve poter aggiornare le variabili ambientali senza perdere la memoria logica del lavoro in corso. A tale scopo è stato progettato lo stato di routing `returnToState` :

```
State returnToState {}
Goto engaged if [# CargoState = "engaged" #] else disengaged
```

Questo meccanismo agisce da trampolino: dopo aver processato l'evento o l'allarme, il sistema valuta la variabile di stato `CargoState` e rientra in `engaged` (se era nel mezzo di un ciclo) o in `disengaged` (se era in attesa), eliminando la ridondanza e impedendo stalli logici.

3.5. 5. Timeout di Sicurezza sul Deposito (`handle_deposit_timeout`)

Per evitare che il sistema rimanga congelato indefinitamente se un cliente, dopo aver ricevuto `load_accepted` , non deposita il container, è stata introdotta una transizione di tempo di 30 secondi nello stato `engaged` . Allo scadere del timeout, l'orchestratore attiva `handle_deposit_timeout` , invia alla stiva un messaggio di rilascio (`free_slot`), spegne il LED e torna allo stato `disengaged` .

4. Confronto Tabellare degli Stati (`cargoservice`)

STATO QAK	SPRINT 0	SPRINT 1	SIGNIFICATO E MODIFICA ARCHITETTURALE
<code>s0</code>	Presente	Presente	Inizializzazione variabili di stato e connessioni.
<code>work</code>	Presente (Attesa)	Eliminato	Sostituito per chiarezza terminologica da due stati distinti.
<code>disengaged</code>	Assente	Nuovo	Sistema libero. Il LED è spento e si attendono richieste.

<code>engaged</code>	Assente	Nuovo	Sistema occupato. Il LED lampeggia, preconditioni attive.
<code>handle_load_request</code>	Solo commenti	Logica eguibile	Es-Valida la guardia booleana e interroga la stiva (<code>get_slot</code>).
<code>wait_for_slot</code>	Assente	Nuovo	Attesa asincrona e non-bloccante della risposta dall'hold.
<code>accept_request</code> / <code>refuse_request</code>	Assenti (in-line)	Nuovi	Smistamento formale delle risposte al cliente (<code>ioport</code>).
<code>handle_sonar</code> / <code>update_service</code>	Assenti	Nuovi	Gestione reattiva degli eventi di presenza e guasto sonar.
<code>returnToState</code>	Assente	Nuovo	Stato di routing per tornare a <code>engaged</code> / <code>disengaged</code> dopo un interrupt.
<code>do_robot_job</code>	Solo commento	Nuovo	Richiesta al robot di prelievo e trasporto verso <code>slot5</code> .
<code>mark_container</code>	Solo commento	Nuovo	Attesa completamento etichettatura da parte del marker.
<code>move_to_reserved</code>	Solo commento	Nuovo	Richiesta di deposito finale nello slot riservato dall'hold.
<code>finish_job</code>	Assente	Nuovo	Chiusura transazione, spegnimento LED, ritorno a <code>disengaged</code> .
<code>handle_deposit_timeout</code>	Assente	Nuovo	Scadenza di 30s. Libera lo slot in stiva (<code>free_slot</code>).

5. Analisi Puntuale del Codice del Prototipo Sprint 1

Per mostrare in modo esaustivo e trasparente come le scelte architetturali discusse siano state concretamente realizzate, di seguito viene riportato l'intero codice QAK del prototipo dello Sprint 1 (`/sprint1/prototype/codice_con_tutti_i_componenti.qak`), analizzato per sezioni logiche (snippet di codice accompagnati da analisi critica).

5.1. 1. Dichiarazione del Sistema, Contesto e Messaggistica

```
System cargosystem

//INTERAZIONI TRA COMPONENTI
```

```

// Customer ↔ CargoService
Request load_request      : loadRequest(none)
Reply   load_accepted     : loadAccepted(SLOTID) for load_request
Reply   load_retrylater   : loadRetryLater(none) for load_request
Reply   load_refused      : loadRefused(none) for load_request

// CargoService ↔ Hold
Request get_slot          : getSlot(none)
Reply   slot_reserved     : slotReserved(SLOTID) for get_slot
Reply   hold_full         : holdFull(none) for get_slot
Dispatch free_slot       : freeSlot(SLOTID)

// Aggiornamenti Sensori e Stato
Event   sonardata         : distance(D) // Emesso dal sonar
Dispatch set_service_status : setServiceStatus(STATUS)
Dispatch led_ctrl         : ledCmd(CMD)

// CargoService ↔ Robot
Request robot_move : robotMove(TARGET)
Reply   robot_done : robotDone(none) for robot_move

// CargoService ↔ MarkerDevice
Request mark_container : markContainer(none)
Reply   marking_done   : markingDone(none) for mark_container

//CONTESTI
Context ctxcargoservice ip [host="localhost" port=8050] // Contesto unico del
prototipo per lo Sprint 1

```

Analisi del Blocco 1: La prima sezione del modello definisce i confini di interazione e l'infrastruttura di comunicazione.

- **Distinzione semantica dei messaggi:** In accordo con le buone pratiche di ingegneria dei protocolli, si distingue rigorosamente tra comunicazioni binarie sincrone/correlate (`Request` / `Reply`), comandi asincroni diretti (`Dispatch` , utilizzati ad esempio per liberare lo slot con `free_slot` o per accendere/spegnere il LED con `led_ctrl`) e notifiche broadcast ambientali (`Event` , impiegato dal sonar per diffondere la misura della distanza `sonardata`).
- **Contesto Unico:** La dichiarazione di `ctxcargoservice` sulla porta `8050` vincola tutti i 7 attori del sistema a eseguire nella stessa JVM. Come argomentato nell'analisi, ciò elimina il rumore infrastrutturale e le latenze TCP durante i test di correttezza della business logic.

5.2. 2. Orchestratore (`cargoservice`): Inizializzazione e Stati di Riposo

```

QActor cargoservice context ctxcargoservice {
  [#
    var IOPortOccupied = false
    var ServiceWorking = true
    var CargoState      = "disengaged"

```

```

        var ReservedSlotId = -1
    #]

    State s0 initial {
        println("cargoservice | STARTED") color magenta
    }
    Goto disengaged

    State disengaged {
        println("cargoservice | DISENGAGED: waiting for requests...") color
blue
    }
    Transition t0
        whenRequest load_request      → handle_load_request
        whenEvent    sonardata         → handle_sonar
        whenMsg       set_service_status → update_service

    State engaged {
        println("cargoservice | ENGAGED: waiting for container deposit...")
color blue
    }
    Transition t0
        whenTime    30000              → handle_deposit_timeout
        whenRequest load_request      → handle_load_request
        whenEvent    sonardata         → handle_sonar
        whenMsg       set_service_status → update_service

```

Analisi del Blocco 2:

- **Variabili del Modello di Memoria:** Nel blocco Kotlin `[# ... #]`, l'attore inizializza le variabili di stato interne che governano la Macchina a Stati Finiti. Il flag `IOPortOccupied` tiene traccia dell'ingombro fisico all'IOPort, `ServiceWorking` memorizza lo stato di salute dell'impianto (interrotto/funzionante), mentre `CargoState` ed esplicitamente il lessico del committente ("engaged/disengaged").
- **Sdoppiamento dell'Attesa:** A differenza dello Sprint 0 dove esisteva un solo stato `work`, qui si osserva la chiara separazione tra `disengaged` (nessuna operazione in corso) ed `engaged` (carico autorizzato e in attesa del container). In entrambi gli stati, l'attore mantiene la reattività verso le richieste di carico, gli eventi del sonar e gli allarmi di guasto.

5.3. 3. Orchestratore (`cargoservice`): Ammissione e Smistamento Richieste

```

//GESTIONE RICHIESTE

State handle_load_request {
    printCurrentMessage color yellow

    // Verifica precondizioni per accettare la richiesta
    if [# CargoState = "engaged" || !ServiceWorking || IOPortOccupied
#] {

```

```

        replyTo load_request with load_retrylater : loadRetryLater(none)
    } else {
        // Interroga la Hold per uno slot libero
        request hold -m get_slot : getSlot(none)
    }
}
Transition t0
    whenReply slot_reserved → accept_request
    whenReply hold_full     → refuse_request

State accept_request {
    onMsg(slot_reserved : slotReserved(ID)) {
        [#
            ReservedSlotId = payloadArg(0).toInt()
            CargoState      = "engaged"
        #]
        forward ledmock -m led_ctrl : ledCmd(blink)
        [# val SlotName = "slot$ReservedSlotId" #]
        replyTo load_request with load_accepted : loadAccepted($SlotName)
    }
}
Goto engaged

State refuse_request {
    replyTo load_request with load_refused : loadRefused(none)
}
Goto disengaged

```

Analisi del Blocco 3:

- **Guardia Booleana di Ammissione:** Lo stato `handle_load_request` implementa la logica di filtro critico dell'intero sistema. La condizione `if [# CargoState = "engaged" || !ServiceWorking || IOPortOccupied #]` valuta simultaneamente la logica di business, la sicurezza hardware e lo stato fisico della pedana. In caso negativo, il servizio emette immediatamente la risposta `load_retrylater` senza bloccare la propria esecuzione.
- **Transazione a due fasi con la Stiva:** Se le precondizioni sono soddisfatte, l'orchestratore non accetta ciecamente la richiesta, ma inoltra una richiesta `get_slot` all'attore `hold`. Solo alla ricezione della risposta asincrona `slot_reserved` (transizione verso `accept_request`), l'orchestratore alloca il `ReservedSlotId`, commuta lo stato in `engaged`, comanda all'attuatore LED di lampeggiare (`ledCmd(blink)`) e invia la conferma `load_accepted(slotX)` al cliente.

5.4. 4. Orchestratore (`cargoservice`): Gestione Sonar, Allarmi e Routing Asincrono

```

//GESTIONE SONAR

State handle_sonar {
    onMsg(sonardata : distance(D)) {

```

```

        [# val Dist = payloadArg(0).toInt() #]

        // D < 50 indica presenza del container (IOPort occupata)
        if [# Dist < 50 #] {
            [# IOPortOccupied = true #]
        } else {
            [# IOPortOccupied = false #]
        }
    }
}

// Se il container viene posato durante lo stato engaged, il robot può
iniziare
Goto do_robot_job if [# IOPortOccupied && CargoState = "engaged" #]
else returnToState

State update_service {
    onMsg(set_service_status : setServiceStatus(S)) {
        [# ServiceWorking = (payloadArg(0) = "working") #]
    }
}

Goto returnToState

State returnToState {}
Goto engaged if [# CargoState = "engaged" #] else disengaged

```

Analisi del Blocco 4:

- **Aggiornamento Dinamico della Scena:** Lo stato `handle_sonar` decodifica il payload dell'evento `distance(D)`. La soglia numerica (`Dist < 50`) mappa il concetto fisico di “presenza del container sulla pedana” nella variabile booleana `IOPortOccupied`.
- **Innesco Reattivo o Routing:** Se la pedana diventa occupata e il sistema si trova nello stato logico di ingaggio (`IOPortOccupied && CargoState = "engaged"`), la FSM scatta automaticamente verso l'avvio della movimentazione robotica (`do_robot_job`).
- **Stato di Routing (`returnToState`):** Costituisce un punto di eccellenza architetturale dello Sprint 1. Sia l'aggiornamento degli allarmi (`update_service`) che le letture sonar non rilevanti terminano con una transizione a `returnToState`. Questo stato privo di corpo agisce da switch logico: interroga `CargoState` e riporta l'attore in `engaged` o in `disengaged`. In questo modo si garantisce resilienza totale alle interruzioni esterne senza duplicare il codice di transizione nei singoli stati operativi.

5.5. 5. Orchestratore (`cargoservice`): Coordinamento Robot, Etichettatura e Timeout

```

// GESTIONE ROBOT E MARKER

State do_robot_job {
    println("cargoservice | Container deposited! Moving to slot5...")
    color magenta
}

```

```

        request cargorobotmock -m robot_move : robotMove(slot5)
    }
    Transition t0
        whenReply robot_done → mark_container

    State mark_container {
        println("cargoservice | At slot5. Asking markerdevice to mark...")
        color magenta
        request markerdevice -m mark_container : markContainer(none)
    }
    Transition t0
        whenReply marking_done → move_to_reserved_slot

    State move_to_reserved_slot {
        println("cargoservice | Marked! Moving to slot$ReservedSlotId...")
        color magenta
        [# val TargetSlot = "slot$ReservedSlotId" #]
        request cargorobotmock -m robot_move : robotMove($TargetSlot)
    }
    Transition t0
        whenReply robot_done → finish_job

    State finish_job {
        println("cargoservice | Job completed!") color green
        [# CargoState = "disengaged" #]
        forward ledmock -m led_ctrl : ledCmd(off)
    }
    Goto disengaged

    State handle_deposit_timeout {
        println("cargoservice | Deposit timeout! Freeing slot.") color red

        [# CargoState = "disengaged" #]
        forward hold -m free_slot : freeSlot($ReservedSlotId)
        forward ledmock -m led_ctrl : ledCmd(off)
    }
    Goto disengaged
}

```

Analisi del Blocco 5:

- **Srotolamento Asincrono (Unrolling):** Questo gruppo di stati incarna il superamento del commento monolitico dello Sprint 0. L'orchestratore governa la sequenza operativa inviando richieste asincrone ai collaboratori (`cargorobotmock` e `markerdevice`). Ogni transizione di avanzamento è strettamente subordinata alla ricezione della risposta di completamento (`whenReply robot_done` , `whenReply marking_done`), garantendo che il robot e il laser operino in sincronia rigorosa senza bloccare il thread dell'orchestratore.
- **Chiusura del Ciclo (`finish_job`):** Una volta depositato il container nello slot finale, l'attore reimposta `CargoState = "disengaged"` e invia il dispatch di spegnimento al LED (`ledCmd(off)`), riportando il sistema allo stato di riposo iniziale.

- **Gestione del Caso Limite (Timeout):** Lo stato di recovery `handle_deposit_timeout` entra in azione allo scadere dei 30 secondi nello stato `engaged` in assenza del container. Per garantire che le risorse non rimangano bloccate a causa dell'inadempienza del cliente, l'orchestratore invia un messaggio di compensazione alla stiva (`freeSlot($ReservedSlotId)`), spegne il LED e risistema l'infrastruttura per nuove richieste.

5.6. 6. Collaboratore Simulato di Stiva (`hold`)

```
//HOLD MOCK

QActor hold context ctxcargoservice {
  [#
    var Slots = intArrayOf(0, 0, 0, 0)

    fun getFreeSlot(): Int {
      for (i in 0..3) {
        if (Slots[i] == 0) return i + 1
      }
      return -1
    }
  #]

  State s0 initial {
    println("hold | STARTED") color green
  }
  Goto work

  State work {}
  Transition t0
    whenRequest get_slot → handle_get_slot
    whenMsg      free_slot → handle_free_slot

  State handle_get_slot {
    [# val SlotId = getFreeSlot() #]

    if [# SlotId ≠ -1 #] {
      [# Slots[SlotId - 1] = 1 #]
      replyTo get_slot with slot_reserved : slotReserved($SlotId)
    } else {
      replyTo get_slot with hold_full : holdFull(none)
    }
  }
  Goto work

  State handle_free_slot {
    onMsg(free_slot : freeSlot(ID)) {
      [# val id = payloadArg(0).toInt() #]
      [# Slots[id - 1] = 0 #]
      println("hold | Freed slot$id") color green
    }
  }
  Goto work
}
```

Analisi del Blocco 6:

- **Memoria Volatile Mappata:** In accordo con le considerazioni sui requisiti ad alta priorità, l'attore `hold` modella la stiva navale tramite una struttura dati in memoria ad alta efficienza (`Slots = intArrayOf(0,0,0,0)`).
- **Atomicità di Allocazione e Rilascio:** Alla ricezione della richiesta `get_slot`, l'attore esegue l'algoritmo lineare di ricerca `getFreeSlot()`. Se individua una cella vuota, ne marca lo stato a `1` e risponde con l'identificativo 1-indexed (`slotReserved`); se tutti i 4 slot sono pieni, risponde immediatamente con `holdFull`. Parallelamente, la gestione del dispatch `free_slot` permette di resettare la cella a `0` (indispensabile sia per il timeout di deposito che per futuri cicli di scarico).

5.7. 7. Sensori e Attuatori Simulati (`sonarmock`, `markerdevice`, `ledmock`)

```
//SONAR MOCK

QActor sonarmock context ctxcargoservice {
    State s0 initial {
        delay 4000
        println("sonarmock | Container placed → Emitting sonardata(30)")
    color cyan
        emit sonardata : distance(30)

        // Simulazione di un guasto al sonar dopo il primo ciclo (D > DFREE
    per 3s)
        delay 8000
        println("sonarmock | Sonar measures D > DFREE for > 3s. System OUT OF
    SERVICE!") color red
        forward cargoservice -m set_service_status :
    setServiceStatus(outofservice)

        // Simulazione del ripristino (D ≤ DFREE)
        delay 5000
        println("sonarmock | Sonar measures D ≤ DFREE. System WORKING
    again!") color green
        forward cargoservice -m set_service_status :
    setServiceStatus(working)
    }
}

//MARKER MOCK

QActor markerdevice context ctxcargoservice {
    State s0 initial {
        println("markerdevice | STARTED") color green
    }
    Goto work

    State work {}
    Transition t0
```

```

        whenRequest mark_container → handle_mark

    State handle_mark {
        println("markerdevice | Marking container...") color cyan
        delay 1500
        println("markerdevice | Container marked!") color cyan
        replyTo mark_container with marking_done : markingDone(none)
    }
    Goto work
}

//LED MOCK

QActor ledmock context ctxcargoservice {
    State s0 initial { }
    Goto work

    State work {}
    Transition t0
        whenMsg led_ctrl → handle_cmd

    State handle_cmd {
        onMsg(led_ctrl : ledCmd(CMD)) {
            println("ledmock | LED is now ${payloadArg(0)}") color green
        }
    }
    Goto work
}

```

Analisi del Blocco 7:

- **Simulatore Sonar Dinamico:** Il `sonarmock` funge da generatore di stimoli per collaudare la reattività dell'orchestratore. Mediante una sequenza temporizzata (`delay`), emette in primo luogo l'evento di presenza container (`distance(30)`), e successivamente simula un guasto ambientale prolungato inviando `setServiceStatus(outofservice)`, seguito dal ripristino (`working`). Ciò dimostra che la FSM è in grado di gestire interrupt dinamici senza fallimenti.
- **Emulazione Attuatori:** Il `markerdevice` simula il tempo fisico di marcatura laser con un ritardo bloccante locale (`delay 1500`) restituendo la risposta di completamento al termine. Il `ledmock` funge da ricevitore puro per certificare visivamente nei log di debug che l'accensione, il lampeggio e lo spegnimento avvengano esattamente in corrispondenza delle transizioni logiche del servizio.

5.8. 8. Client e Robot Simulati (`ioportmock`, `cargorobotmock`)

```

//IOPORT MOCK

QActor ioportmock context ctxcargoservice {
    State s0 initial {
        delay 1000
    }
}

```

```

        println("ioportmock | Sending 1st load_request (should be accepted)")
color cyan
        request cargospace -m load_request : loadRequest(none)
    }
    Transition t0
        whenReply load_accepted → handle_accept
        whenReply load_retrylater → handle_retry
        whenReply load_refused → handle_refuse

    State handle_accept {
        printCurrentMessage color green

        // Aspettiamo che il sonarmock vada in outofservice (circa al ms
12000)
        delay 12000
        println("ioportmock | Sending 2nd load_request (should retrylater due
to outofservice)") color cyan
        request cargospace -m load_request : loadRequest(none)
    }
    Transition t0
        whenReply load_retrylater → wait_and_retry
        whenReply load_accepted → handle_final_accept
        whenReply load_refused → handle_refuse

    State wait_and_retry {
        printCurrentMessage color yellow

        // Aspettiamo il ripristino del sonarmock (circa al ms 17000)
        delay 6000
        println("ioportmock | Sending 3rd load_request (should be accepted,
system recovered)") color cyan
        request cargospace -m load_request : loadRequest(none)
    }
    Transition t0
        whenReply load_accepted → handle_final_accept
        whenReply load_retrylater → handle_retry
        whenReply load_refused → handle_refuse

    State handle_final_accept { printCurrentMessage color green }
    State handle_retry { printCurrentMessage color yellow }
    State handle_refuse { printCurrentMessage color red }
}

//CARGOROBOT MOCK

QActor cargorobotmock context ctxcargospace {
    State s0 initial { }
    Goto work

    State work {}
    Transition t0
        whenRequest robot_move → handle_move

    State handle_move {

```

```

        delay 1500
        replyTo robot_move with robot_done : robotDone(none)
    }
    Goto work
}

```

Analisi del Blocco 8:

- **Scripting di Collaudo Integrato (`ioportmock`)**: Questo attore rappresenta un client intelligente che esegue uno script di collaudo end-to-end all'avvio del sistema. Invia 3 richieste in momenti strategici del ciclo di vita del prototipo: la prima trova il sistema libero ed operante (ricevendo `load_accepted`); la seconda viene inviata al millisecondo 12000, esattamente durante la simulazione di guasto del sonar (certificando che l'orchestratore risponda correttamente con `load_retrylater`); la terza avviene dopo il ripristino, confermando che il sistema è pienamente resiliente ed in grado di accogliere nuove richieste una volta risolta l'anomalia.
- **Astrazione delle Movimentazioni (`cargorobotmock`)**: Il robot simulato risponde al contratto di movimentazione (`robotMove`) con un tempo d'attesa parametrico (`delay 1500`), separando in modo netto le responsabilità tra chi decide dove deve andare il container (l'orchestratore) e chi esegue materialmente il moto (il robot, la cui navigazione su mappa verrà approfondita con il `basicrobot` nello Sprint successivo).

6. Piano di Test e Collaudo Automatizzato

L'impiego di QAK e la scomposizione modulare realizzata nello Sprint 1 forniscono la base per la verifica rigorosa dei requisiti. La suite di collaudo, tradotta in codice Kotlin con framework JUnit (`TestCargoServiceCore.kt`), agisce come un client esterno che si connette al contesto `ctxcargoservice` (porta 8050) e invia richieste strutturate secondo la semantica Prolog (`msg(load_request, ...)`).

L'esecuzione nel contesto unico garantisce un collaudo ripetibile e deterministico:

1. **Verifica Accettazione (T1)**: A stiva disponibile e sistema `disengaged` , il test verifica l'effettiva ricezione della risposta `load_accepted` e la corretta prenotazione dello slot.
2. **Verifica Rinvio e Rifiuto (T2/T3)**: Simula condizioni di stiva satura o pedana occupata, asserendo il ritorno di `load_retrylater` o `load_refused` .
3. **Resilienza agli Allarmi**: Verifica che l'invio di un dispatch `set_service_status(outofservice)` inneschi il rinvio delle richieste e che la successiva risoluzione (`working`) ripristini la normale operatività tramite la transizione `returnToState` .

7. Conclusioni e Sviluppi Futuri

In sintesi, le scelte progettuali dello Sprint 1 sono state guidate da criteri di modularità, chiarezza comportamentale e aderenza al lessico di dominio. L'architettura formalizzata

garantisce la totale separazione tra la logica di controllo e le dipendenze fisiche esterne. Il prototipo così validato costituisce la base solida ed estensibile per gli sviluppi dello Sprint 2, nei quali i collaboratori mock verranno progressivamente sostituiti dalle interfacce reali verso la web-gui, i sensori su Raspberry Pi e l'attuatore `cargorobot`.